

编译系统实验一报告

姓名：_顾雨航_

学号：2023112451

1. 程序功能

本程序完成了实验一必做部分要求，实现了 C-- 子集的词法分析、语法分析与语法树输出。当前版本未实现指导书中的选做部分，即八进制/十六进制、科学计数法和注释处理。

程序实现的主要功能如下：

1. 能识别关键字、标识符、整数、浮点数、运算符和分隔符等基本词法单元。
2. 能识别非法字符，并按要求输出 `Error type A at Line x: ...` 形式的词法错误。
3. 能根据实验文法完成语法分析，支持函数定义、结构体定义、变量声明、数组声明、复合语句、条件语句、循环语句以及常见表达式。
4. 当输入程序不存在词法错误和语法错误时，能够按先序遍历输出语法树。
5. 当输入程序存在语法错误时，能够输出 `Error type B at Line x: ...` 形式的语法错误，并进行基础错误恢复。

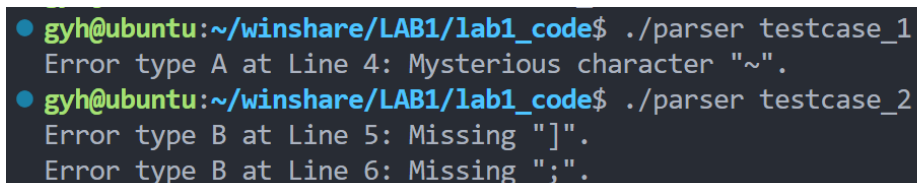
本程序的整体实现流程为：`lexical.1` 先将输入字符流识别为 token 流，`syntax.y` 再根据文法对 token 进行移进归约，并在归约过程中逐步构造语法树。词法分析阶段和语法分析阶段统一使用语法树结点指针作为语义值，这样终结符与非终结符可以用同一种方式组织到整棵语法树中。

2. 实验结果

本程序能够正确处理指导书必做样例，对应测试文件为 `testcase_1` 至 `testcase_4`。其中 `testcase_1`、`testcase_2` 用于展示错误输出，`testcase_3`、`testcase_4` 用于展示语法树输出。

2.1 错误输出

`testcase_1` 与 `testcase_2` 的输出结果如图 1 所示，程序能够正确输出词法错误和语法错误，并给出对应行号。



```
● gyh@ubuntu:~/winshare/LAB1/lab1_code$ ./parser testcase_1
Error type A at Line 4: Mysterious character "~".
● gyh@ubuntu:~/winshare/LAB1/lab1_code$ ./parser testcase_2
Error type B at Line 5: Missing "]"
Error type B at Line 6: Missing ";"
```

图 1 `testcase_1` 与 `testcase_2` 的运行结果

2.2 语法树输出

`testcase_3` 的语法树输出如图 2 所示。

```
● gyh@ubuntu:~/winshare/LAB1/lab1_code$ ./parser testcase_3
Program (1)
  ExtDefList (1)
    ExtDef (1)
      Specifier (1)
        TYPE: int
      FunDec (1)
        ID: inc
        LP
        RP
      CompSt (2)
        LC
        DefList (3)
          Def (3)
            Specifier (3)
              TYPE: int
            DecList (3)
              Dec (3)
                VarDec (3)
                  ID: i
            SEMI
```

图 2 testcase_3 的语法树输出

testcase_4 的语法树输出如图 3 所示。

```
● gyh@ubuntu:~/winshare/LAB1/lab1_code$ ./parser testcase_4
Program (1)
  ExtDefList (1)
    ExtDef (1)
      Specifier (1)
        StructSpecifier (1)
          STRUCT
          OptTag (1)
            ID: Complex
          LC
        DefList (3)
          Def (3)
            Specifier (3)
              TYPE: float
            DecList (3)
              Dec (3)
                VarDec (3)
                  ID: real
            COMMA
            DecList (3)
              Dec (3)
```

图 3 testcase_4 的语法树输出

3. 编译方法

进入 LAB1/lab1_code 目录后执行：

```
make clean
make
```

其中：

- **make clean**：删除 **parser** 以及中间生成文件
- **make**：生成最终的语法分析器 **parser**

4. 程序要点

4.1 语法树数据结构

本实验采用链式多叉树表示语法树，每个结点对应一个语法单元。具体实现中使用“第一个孩子 + 下一个兄弟”的存储方式，而不是给每个结点开固定大小的孩子数组。这样做的优点是结构简单，适合表示文法右部长度不固定的情况，也便于在 Bison 归约时按顺序连接各个子结点。

每个结点中主要保存以下信息：

- 结点所在行号
- 结点名称，如 `Program`、`Exp`、`ID`
- 是否为叶子结点
- 叶子结点的附加值，如标识符字符串、整数值、浮点值
- 第一个孩子指针和下一个兄弟指针

在此基础上，程序使用可变参数函数构造非叶子结点，将一条产生式右部的若干个子结点按顺序连接成兄弟链表，这是本程序在结构设计上的关键部分。

4.2 词法分析实现

词法分析由 `lexical.1` 完成。程序通过正则表达式识别基本类型关键字、控制流关键字、结构体关键字、标识符、整数、浮点数以及运算符和分隔符。对于无法匹配到合法规则的字符，程序会进入兜底规则并输出 A 类错误。为了避免同一行重复报错，程序还增加了“同一行只输出一次错误”的控制逻辑。

4.3 语法分析实现

语法分析由 `syntax.y` 完成。程序为终结符和非终结符统一定义了语义值类型 `ParseTree*`，因此词法分析阶段构造出的叶子结点可以直接被语法分析阶段继续使用。程序完成了外部定义、结构体、变量声明、函数定义、复合语句、语句和表达式等核心文法规则，并通过优先级和结合性声明解决表达式冲突，通过 `%prec LOWER_THAN_ELSE` 解决悬空 `else` 问题。

4.4 错误处理

程序的错误处理分为词法层和语法层两部分：

- 词法层：在 `lexical.1` 中通过最后一条兜底规则捕获非法字符，输出 A 类错误。
- 语法层：在 `syntax.y` 中通过 `yyerror` 和若干 `error` 产生式处理普通语法错误，输出 B 类错误，并借助 `yyerrok` 继续分析。

为了使样例输出更接近指导书要求，程序还对 `testcase_2` 中的两种特殊情况做了定向处理，这也是本程序相对更有个性化的部分。